

Analyzing Crop Production Across India

India, an agrarian economy, has witnessed significant transformations in its agricultural patterns due to factors such as urbanization, climate change, and technological advances. Analyzing these shifts at the district level provides insights into food security, resource allocation, and sustainable farming practices.

However, while existing studies extensively explore macro-level trends (state or national scale), granular district-level insights over extended timeframes (1997–2023) remain underexplored. This project aims at trying to explore this segment in Indian crop production

For context Warangal is the city I spent most of my first 12 years in. Its the city where most of my relatives live in. Since its very close to my heart thats the city I'm going to be using in all of the examples

```
import tensorflow as tf
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from matplotlib.patches import Patch
import scipy.stats as stats
import seaborn as sns
import geopandas as gpd

import folium
from folium import plugins
import branca.colormap as cm

from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error,
r2_score
from sklearn.preprocessing import OneHotEncoder, MinMaxScaler,
StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.cluster import KMeans

from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor
from sklearn.tree import plot_tree
from sklearn.model_selection import cross_val_score
from sklearn.metrics import accuracy_score

from sklearn.decomposition import PCA

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout

APY = pd.read_csv("APY.csv")
rainfall = pd.read_csv("rainfall.csv")
```

```

temp = pd.read_csv("temperature.csv")
agri_pricing = pd.read_csv("agr_csv2.csv")
india_map = gpd.read_file('States/India_State_Boundary.shp')
india_dt = gpd.read_file('India Shape/india_ds.shp')

APY_dict = {
    'APY': APY,
    'rainfall': rainfall,
    'temp': temp,
    'agri_pricing': agri_pricing
}

for i, j in APY_dict.items():
    j.dropna(axis=0)
    print(f"{i}'s shape is {j.shape}")

india_dt.columns.tolist()

APY's shape is (345336, 8)
rainfall's shape is (58, 18)
temp's shape is (58, 6)
agri_pricing's shape is (2238, 9)

['DST_ID', 'DISTRICT', 'STATE', 'geometry']

```

Start off with creating choropleth maps of production of crops per state

```

# mapping the statenames in APY to shapefile so that there's
consistency
STATE_NAME_MAPPING = {
    'Andaman and Nicobar Island': 'Andaman & Nicobar',
    'Chhattisgarh': 'Chhattishgarh',
    'Dadra and Nagar Haveli': 'Daman and Diu and Dadra and Nagar
Haveli',
    'THE DADRA AND NAGAR HAVELI': 'Daman and Diu and Dadra and Nagar
Haveli',
    'Daman and Diu': 'Daman and Diu and Dadra and Nagar Haveli',
    'Tamil Nadu': 'Tamilnadu',
    'Telangana': 'Telengana',
    'Laddak': 'Ladakh',
    'CHANDIGARH': 'Chandigarh'
}

# Calculate the Total Production of each state and plot it

APY_standardized = APY.copy()
APY_standardized['State'] =
APY_standardized['State'].replace(STATE_NAME_MAPPING)

```

```

total_production = APY_standardized.groupby(['State', 'Crop'])
['Production'].sum().reset_index()

dominant_crops = total_production.loc[
    total_production.groupby('State')['Production'].idxmax()
]

dominant_crops = dominant_crops.sort_values('Production',
ascending=False)

fig = plt.figure(figsize=(20, 15))
gs = plt.GridSpec(1, 2, width_ratios=[2, 1]) # Adjust ratio for map
and legend
ax_map = fig.add_subplot(gs[0])
ax_legend = fig.add_subplot(gs[1])

# Create color mapping
unique_crops = dominant_crops['Crop'].unique()
colors = plt.cm.Set3(np.linspace(0, 1, len(unique_crops)))
crop_colors = dict(zip(unique_crops, colors))

# Merge with geographic data
merged_data = india_map.merge(
    dominant_crops,
    how='left',
    left_on='State_Name',
    right_on='State'
)

# Plot base map
merged_data.plot(
    ax=ax_map,
    color='lightgray',
    edgecolor='black',
    linewidth=0.5
)

# Plot states with data
for crop in unique_crops:
    crop_states = merged_data[merged_data['Crop'] == crop]
    if not crop_states.empty:
        crop_states.plot(
            ax=ax_map,
            color=crop_colors[crop],
            edgecolor='black',
            linewidth=0.5
        )

# Create detailed legend
ax_legend.axis('off')

```

```

y_position = 1
line_height = 0.03

# Group states by crop
for crop in unique_crops:
    crop_data = dominant_crops[dominant_crops['Crop'] == crop]

    # Add crop header
    ax_legend.add_patch(mpatches.Rectangle((0.1, y_position), 0.1,
line_height,
                                     facecolor=crop_colors[crop]))
    ax_legend.text(0.25, y_position, f'{crop}', fontsize=12,
fontWeight='bold')
    y_position -= line_height * 1.5

    # Add states for this crop
    for _, row in crop_data.iterrows():
        text = f"{row['State']}: {row['Production']:.0f} tonnes"
        ax_legend.text(0.25, y_position, text, fontsize=10)
        y_position -= line_height * 1.2

    y_position -= line_height # Extra space between crops

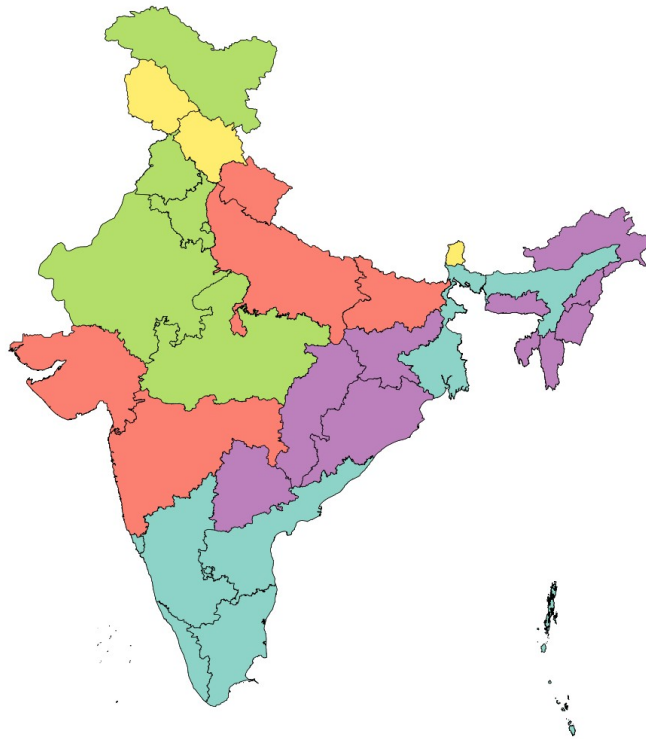
# Add "No Data" entry
if len(merged_data[merged_data['Crop'].isna()]) > 0:
    ax_legend.add_patch(mpatches.Rectangle((0.1, y_position), 0.1,
line_height,
                                     facecolor='lightgray'))
    ax_legend.text(0.25, y_position, 'No Data:', fontsize=12,
fontWeight='bold')
    y_position -= line_height * 1.5

    no_data_states = merged_data[merged_data['Crop'].isna()]
    ['State_Name'].unique()
    for state in sorted(no_data_states):
        ax_legend.text(0.25, y_position, state, fontsize=10)
        y_position -= line_height * 1.2

plt.suptitle('Dominant Crop by Total Production (Tonnes)', y=0.95,
fontSize=16)
ax_map.axis('off')
plt.show()

```

Dominant Crop by Total Production (Tonnes)



Coconut

Kerala: 129,607,125,000 tonnes
Tamilnadu: 77,024,400,000 tonnes
Karnataka: 62,692,166,000 tonnes
Andhra Pradesh: 25,212,327,031 tonnes
West Bengal: 8,068,332,400 tonnes
Assam: 3,455,431,000 tonnes
Goa: 2,188,980,000 tonnes
Andaman & Nicobar: 2,052,310,000 tonnes
Puducherry: 486,987,000 tonnes

Sugarcane

Uttar Pradesh: 3,062,485,896 tonnes
Maharashtra: 1,396,763,558 tonnes
Gujarat: 294,470,104 tonnes
Bihar: 178,942,980 tonnes
Uttarakhand: 137,657,488 tonnes
Daman and Diu and Dadra and Nagar Haveli: 1,257,785 tonnes

Wheat

Punjab: 364,370,000 tonnes
Madhya Pradesh: 288,882,663 tonnes
Haryana: 241,279,000 tonnes
Rajasthan: 196,477,287 tonnes
Delhi: 1,915,317 tonnes
Chandigarh: 62,748 tonnes
Ladakh: 47,748 tonnes

Rice

Odisha: 153,257,182 tonnes
Chhattishgarh: 116,274,060 tonnes
Telengana: 45,590,609 tonnes
Jharkhand: 28,771,602 tonnes
Tripura: 14,710,040 tonnes
Manipur: 9,426,598 tonnes
Nagaland: 7,818,760 tonnes
Meghalaya: 5,169,434 tonnes
Arunachal Pradesh: 3,851,550 tonnes
Mizoram: 1,719,535 tonnes

Maize

Himachal Pradesh: 14,003,230 tonnes
Jammu and Kashmir: 9,999,452 tonnes
Sikkim: 1,664,559 tonnes

No Data:

Lakshadweep

Now implement the production per hectare

Calculate the Total Production/Hectare of each state and plot it

```

APY_standardized = APY.copy()
APY_standardized['State'] =
APY_standardized['State'].replace(STATE_NAME_MAPPING)

# Calculate yield for each state-crop
state_yields = APY_standardized.groupby(['State', 'Crop']).agg({
    'Production': 'sum',
    'Area ': 'sum'
}).reset_index()
state_yields['Yield'] = state_yields['Production'] /
state_yields['Area ']

# Find crop with maximum yield for each state
dominant_yields = state_yields.loc[
    state_yields.groupby('State')['Yield'].idxmax()
]

# Sort states by yield for legend
dominant_yields = dominant_yields.sort_values('Yield',
ascending=False)

# Create visualization
fig = plt.figure(figsize=(20, 15))
gs = plt.GridSpec(1, 2, width_ratios=[2, 1]) # Adjust ratio for map
and legend
ax_map = fig.add_subplot(gs[0])
ax_legend = fig.add_subplot(gs[1])

# Create color mapping
unique_crops = dominant_yields['Crop'].unique()
colors = plt.cm.Set3(np.linspace(0, 1, len(unique_crops)))
crop_colors = dict(zip(unique_crops, colors))

# Merge with geographic data
merged_data = india_map.merge(
    dominant_yields,
    how='left',
    left_on='State_Name',
    right_on='State'
)

# Plot base map
merged_data.plot(
    ax=ax_map,
    color='lightgray',
    edgecolor='black',
    linewidth=0.5
)

# Plot states with data

```

```

for crop in unique_crops:
    crop_states = merged_data[merged_data['Crop'] == crop]
    if not crop_states.empty:
        crop_states.plot(
            ax=ax_map,
            color=crop_colors[crop],
            edgecolor='black',
            linewidth=0.5
        )

# Create detailed legend
ax_legend.axis('off')
y_position = 1
line_height = 0.03

# Group states by crop
for crop in unique_crops:
    crop_data = dominant_yields[dominant_yields['Crop'] == crop]

    # Add crop header
    ax_legend.add_patch(mpatches.Rectangle((0.1, y_position), 0.1,
                                            line_height,
                                            facecolor=crop_colors[crop]))
    ax_legend.text(0.25, y_position, f'{crop}', fontsize=12,
                  fontweight='bold')
    y_position -= line_height * 1.5

    # Add states for this crop
    for _, row in crop_data.iterrows():
        text = f"{row['State']}: {row['Yield']:.2f} tonnes/ha"
        ax_legend.text(0.25, y_position, text, fontsize=10)
        y_position -= line_height * 1.2

    y_position -= line_height # Extra space between crops

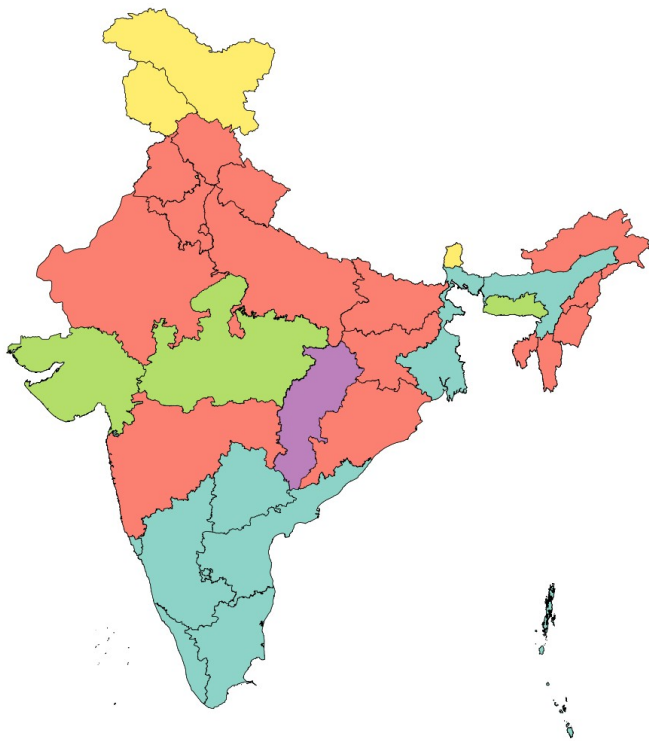
# Add "No Data" entry
if len(merged_data[merged_data['Crop'].isna()]) > 0:
    ax_legend.add_patch(mpatches.Rectangle((0.1, y_position), 0.1,
                                            line_height,
                                            facecolor='lightgray'))
    ax_legend.text(0.25, y_position, 'No Data:', fontsize=12,
                  fontweight='bold')
    y_position -= line_height * 1.5

    no_data_states = merged_data[merged_data['Crop'].isna()]
    ['State_Name'].unique()
    for state in sorted(no_data_states):
        ax_legend.text(0.25, y_position, state, fontsize=10)
        y_position -= line_height * 1.2

```

```
plt.suptitle('Dominant Crop by Yield (Tonnes per Hectare)', y=0.95,  
            fontsize=16)  
ax_map.axis('off')  
plt.show()
```


Dominant Crop by Yield (Tonnes per Hectare)



Coconut

- Telangana: 18710.27 tonnes/ha
- Andhra Pradesh: 12826.95 tonnes/ha
- West Bengal: 12534.68 tonnes/ha
- Puducherry: 12070.87 tonnes/ha
- Tamilnadu: 11969.70 tonnes/ha
- Assam: 7779.05 tonnes/ha
- Karnataka: 6764.74 tonnes/ha
- Kerala: 6742.65 tonnes/ha
- Goa: 5044.84 tonnes/ha
- Andaman & Nicobar: 4229.55 tonnes/ha
- Daman and Diu and Dadra and Nagar Haveli: 300.18 tonnes/ha

Sugarcane

- Maharashtra: 81.28 tonnes/ha
- Delhi: 67.66 tonnes/ha
- Punjab: 65.71 tonnes/ha
- Uttar Pradesh: 62.75 tonnes/ha
- Uttarakhand: 62.33 tonnes/ha
- Odisha: 61.65 tonnes/ha
- Haryana: 60.89 tonnes/ha
- Manipur: 56.27 tonnes/ha
- Rajasthan: 56.25 tonnes/ha
- Bihar: 52.42 tonnes/ha
- Tripura: 50.33 tonnes/ha
- Nagaland: 44.14 tonnes/ha
- Jharkhand: 40.39 tonnes/ha
- Arunachal Pradesh: 19.78 tonnes/ha
- Mizoram: 16.04 tonnes/ha
- Himachal Pradesh: 14.92 tonnes/ha

Banana

- Gujarat: 69.59 tonnes/ha
- Madhya Pradesh: 54.94 tonnes/ha
- Meghalaya: 12.09 tonnes/ha

Bajra

- Chhattishgarh: 25.72 tonnes/ha

Potato

- Chandigarh: 23.06 tonnes/ha
- Jammu and Kashmir: 10.03 tonnes/ha
- Ladakh: 9.33 tonnes/ha
- Sikkim: 4.57 tonnes/ha

No Data:

- Lakshadweep

Implementing K-NN to figure out what parts of India grow what major crop

First lets group the data by district and crop so that each row is a district and each column is a crop and the entries are production amounts

And then scales the data to create target variable (y) with the dominant crop for each district

```
# Create feature matrix
# First summarize production by district and crop
APY_knn = APY.copy()
district_crop_pivot = APY_knn.pivot_table(
    index='District',
    columns='Crop',
    values='Production',
    aggfunc='sum',
    fill_value=0
)

# Create target variable (dominant crop for each district)
district_crop_summary = APY_knn.groupby(['District', 'Crop'])
['Production'].sum().reset_index()
dominant_crops = district_crop_summary.loc[
    district_crop_summary.groupby('District')['Production'].idxmax()
]

scaler = StandardScaler()
X = scaler.fit_transform(district_crop_pivot)
y = dominant_crops['Crop']

# Print summary statistics
print("\n=== Dataset Summary ===")
print(f"Number of districts: {len(district_crop_pivot)}")
print(f"Number of crops (features): {len(district_crop_pivot.columns)}")
print(f"Number of unique target crops: {len(y.unique())}")

=== Dataset Summary ===
Number of districts: 707
Number of crops (features): 55
Number of unique target crops: 21

# Get prepared data using prepare_for_knn
district_names = district_crop_pivot.index

# Apply K-means clustering
n_clusters = 5
kmeans = KMeans(n_clusters=n_clusters, random_state=42)
```

```

labels = kmeans.fit_predict(X)

# Apply PCA for visualization
pca = PCA(n_components=2)
pca_result = pca.fit_transform(X)

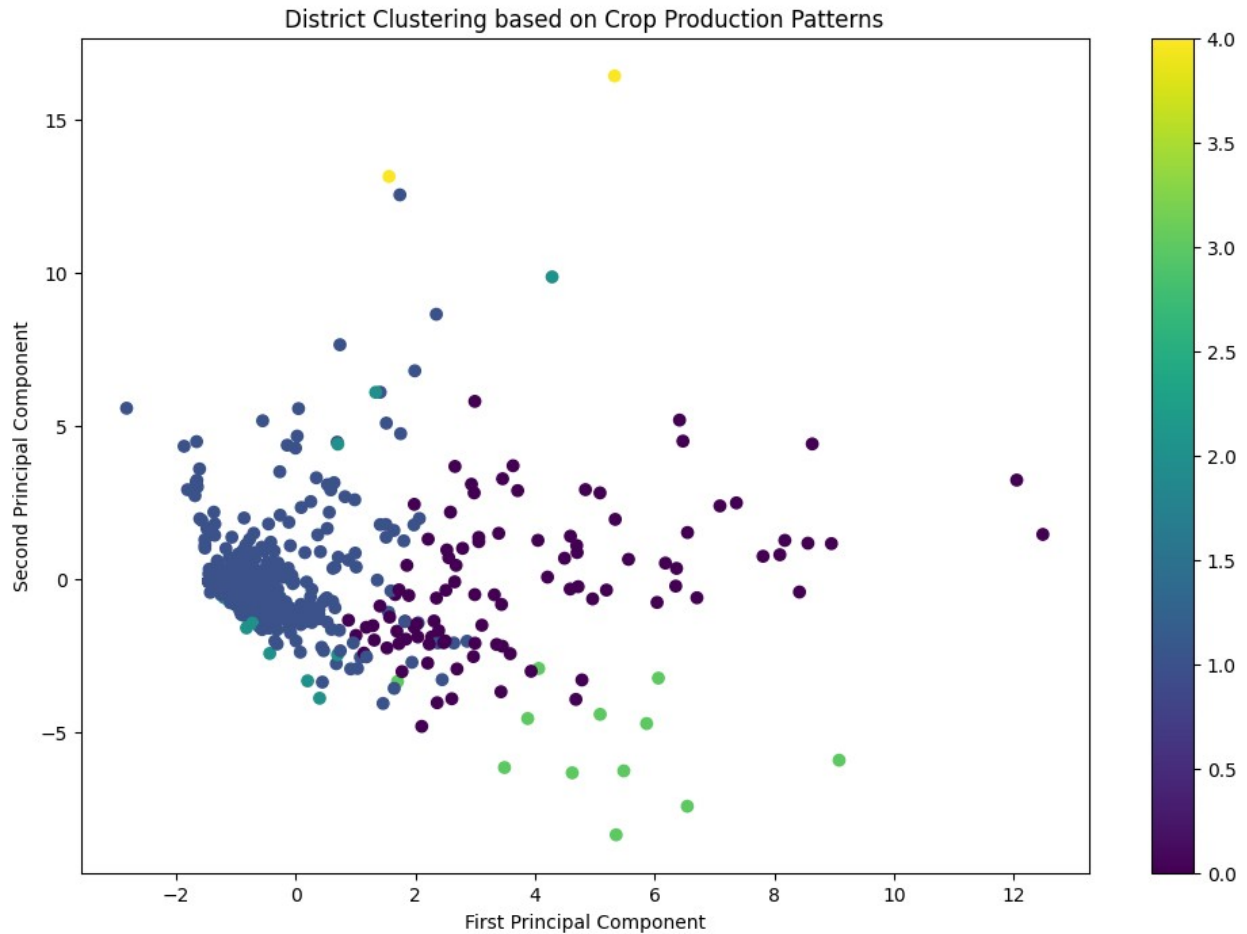
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/
site-packages/sklearn/cluster/_kmeans.py:870: FutureWarning: The
default value of `n_init` will change from 10 to 'auto' in 1.4. Set
the value of `n_init` explicitly to suppress the warning
    warnings.warn(

# Create visualization
plt.figure(figsize=(12, 8))
scatter = plt.scatter(pca_result[:, 0], pca_result[:, 1], c=labels,
cmap='viridis')
plt.title('District Clustering based on Crop Production Patterns')
plt.xlabel('First Principal Component')
plt.ylabel('Second Principal Component')
plt.colorbar(scatter)
plt.show()

# Create cluster DataFrame
cluster_APY = pd.DataFrame({
    'District': district_names,
    'Cluster': labels,
    'Dominant_Crop': y
})

# Create district crop summary for analysis
district_crop_summary = APY.groupby(['District', 'Crop'])
['Production'].sum().reset_index()

```



```
# Analyze clusters
print("\n=== Cluster Analysis ===")
for cluster in range(n_clusters):
    cluster_districts = cluster_APY[cluster_APY['Cluster'] == cluster]
    ['District']
    cluster_crops = district_crop_summary[
        district_crop_summary['District'].isin(cluster_districts)
    ]

    print(f"\nCluster {cluster} characteristics:")
    print("Number of districts:", len(cluster_districts))
    print("Top 5 crops by production:")
    print(cluster_crops.groupby('Crop')['Production']
          .sum()
          .sort_values(ascending=False)
          .head()
    )
    print("\nDistricts in this cluster:")
    print(cluster_districts.tolist())

# Print cluster summary
```

```

print("\n=== Cluster Summary ===")
print("\nNumber of districts in each cluster:")
print(cluster_APY['Cluster'].value_counts())

print("\nDominant crops by cluster:")
print(cluster_APY.groupby('Cluster')['Dominant_Crop'].value_counts())

```

=== Cluster Analysis ===

Cluster 0 characteristics:
 Number of districts: 98
 Top 5 crops by production:

Crop	
Coconut	8.404252e+09
Sugarcane	2.119517e+09
Wheat	3.533353e+08
Cotton(lint)	2.926095e+08
Rice	2.285058e+08

Name: Production, dtype: float64

Districts in this cluster:

['ADILABAD', 'AHMEDNAGAR', 'AKOLA', 'AMRAVATI', 'AMRELI', 'ANANTAPUR', 'ASHOKNAGAR', 'AURANGABAD', 'BAGALKOTE', 'BALAGHAT', 'BALLARI', 'BANDA', 'BARAN', 'BEED', 'BELAGAVI', 'BETUL', 'BHAVNAGAR', 'BIDAR', 'BULDHANA', 'CHANDRAPUR', 'CHHATARPUR', 'CHHINDWARA', 'CHITRADURGA', 'CHITTORGARH', 'DAMOH', 'DEWAS', 'DHAR', 'DHARWAD', 'DHULE', 'GADAG', 'GUNA', 'GUNTUR', 'HAMIRPUR', 'HAVERI', 'HINGOLI', 'HOSHANGABAD', 'INDORE', 'JABALPUR', 'JALAUN', 'JALGAON', 'JALNA', 'JAMNAGAR', 'JHALAWAR', 'JHANSI', 'JUNAGADH', 'KADAPA', 'KALABURAGI', 'KARIMNAGAR', 'KHARGONE', 'KOLHAPUR', 'KOPPAL', 'KOTA', 'KRISHNA', 'KURNOOL', 'LALITPUR', 'LATUR', 'MAHBUBNAGAR', 'MAHOBA', 'MANDSAUR', 'MEDAK', 'NAGPUR', 'NANDED', 'NANDURBAR', 'NARSINGHPUR', 'NASHIK', 'NEEMUCH', 'NIZAMABAD', 'OSMANABAD', 'PANNA', 'PARBHANI', 'PRAKASAM', 'PUNE', 'RAICHUR', 'RAISEN', 'RAJGARH', 'RAJKOT', 'RATLAM', 'REWA', 'SAGAR', 'SANGLI', 'SATARA', 'SATNA', 'SEHORE', 'SEONI', 'SHAJAPUR', 'SHIVPURI', 'SIDHI', 'SOLAPUR', 'SURENDRANAGAR', 'TIKAMGARH', 'UJJAIN', 'VADODARA', 'VIDISHA', 'VIJAYAPURA', 'WARANGAL', 'WARDHA', 'WASHIM', 'YAVATMAL']

Cluster 1 characteristics:
 Number of districts: 581
 Top 5 crops by production:

Crop	
Coconut	2.683292e+11
Sugarcane	4.971582e+09
Rice	1.664648e+09
Wheat	1.549058e+09
Potato	3.983404e+08

Name: Production, dtype: float64

Districts in this cluster:

['AGAR MALWA', 'AGRA', 'AHMADABAD', 'AIZAWL', 'AJMER', 'ALAPPUZHA', 'ALIGARH', 'ALIPURDUAR', 'ALIRAJPUR', 'ALLAHABAD', 'ALMORA', 'AMBALA', 'AMBEDKAR NAGAR', 'AMETHI', 'AMRITSAR', 'AMROHA', 'ANAND', 'ANANTNAG', 'ANJAW', 'ANUGUL', 'ANUPPUR', 'ARARIA', 'ARAVALLI', 'ARIYALUR', 'ARWAL', 'AURAIYA', 'AZAMGARH', 'BADGAM', 'BAGESHWAR', 'BAGHPAT', 'BAHRAICH', 'BAKSA', 'BALANGIR', 'BALESHWAR', 'BALLIA', 'BALOD', 'BALODA BAZAR', 'BALRAMPUR', 'BANDIPORA', 'BANGALORE RURAL', 'BANKA', 'BANSWARA', 'BARABANKI', 'BARAMULLA', 'BAREILLY', 'BARGARH', 'BARNALA', 'BARPETA', 'BARWANI', 'BASTAR', 'BASTI', 'BATHINDA', 'BEGUSARAI', 'BEMETARA', 'BENGALURU URBAN', 'BHADRADRI', 'BHADRAK', 'BHAGALPUR', 'BHANDARA', 'BHARATPUR', 'BHARUCH', 'BHILWARA', 'BHIND', 'BHIWANI', 'BHOJPUR', 'BHOPAL', 'BIJAPUR', 'BIJNOR', 'BILASPUR', 'BISHNUPUR', 'BISWANATH', 'BOKARO', 'BONGAIGAON', 'BOTAD', 'BOUDH', 'BUDAUN', 'BULANDSHAHR', 'BUNDI', 'BURHANPUR', 'BUXAR', 'CACHAR', 'CHAMARAJANAGARA', 'CHAMBA', 'CHAMOLI', 'CHAMPAWAT', 'CHAMPHAI', 'CHANDAUJI', 'CHANDEL', 'CHANDIGARH', 'CHANGLANG', 'CHARAIDE', 'CHARKI DADRI', 'CHATRA', 'CHENGALPATTU', 'CHENNAI', 'CHHOTAUDEPUR', 'CHIKKABALLAPURA', 'CHIKKAMAGALURU', 'CHIRANG', 'CHITRAKOOT', 'CHITTOOR', 'CHURACHANDPUR', 'COIMBATORE', 'CUDDALORE', 'CUTTACK', 'DADRA AND NAGAR HAVELI', 'DAKSHINA KANNADA', 'DAMAN', 'DANG', 'DANTEWADA', 'DARBHANGA', 'DARJEELING', 'DARRANG', 'DATIA', 'DAUSA', 'DAVANGERE', 'DEHRADUN', 'DELHI TOTAL', 'DEOGARH', 'DEOGHAR', 'DEORIA', 'DEVBHUMI DWARKA', 'DHALAI', 'DHAMTARI', 'DHANBAD', 'DHARMAPURI', 'DHEMAJI', 'DHENKANAL', 'DHOLPUR', 'DHUBRI', 'DIBANG VALLEY', 'DIBRUGARH', 'DIMA HASAO', 'DIMAPUR', 'DINAJPUR DAKSHIN', 'DINDIGUL', 'DINDORI', 'DIU', 'DODA', 'DOHAD', 'DUMKA', 'DUNGARPUR', 'DURG', 'EAST DISTRICT', 'EAST GARO HILLS', 'EAST JAINTIA HILLS', 'EAST KAMENG', 'EAST KHASI HILLS', 'EAST SIANG', 'EAST SINGHBUM', 'ERNAKULAM', 'ERODE', 'ETAH', 'ETAWAH', 'FAIZABAD', 'FARIDABAD', 'FARIDKOT', 'FARRUKHABAD', 'FATEHABAD', 'FATEHGARH SAHIB', 'FATEHPUR', 'FAZILKA', 'FIROZABAD', 'FIROZEPUR', 'GADCHIROLI', 'GAJAPATI', 'GANDERBAL', 'GANDHINAGAR', 'GANJAM', 'GARHWA', 'GARIYABAND', 'GAURELLA-PENDRA-MARWAHI', 'GAUTAM BUDDHA NAGAR', 'GAYA', 'GHAZIABAD', 'GHAZIPUR', 'GIR SOMNATH', 'GIRIDIH', 'GOALPARA', 'GODDA', 'GOLAGHAT', 'GOMATI', 'GONDA', 'GONDIA', 'GOPALGANJ', 'GORAKHPUR', 'GUMLA', 'GURDASPUR', 'GURGAON', 'GWALIOR', 'HAILAKANDI', 'HANUMAKONDA', 'HAPUR', 'HARDA', 'HARDOI', 'HARIDWAR', 'HATHRAS', 'HAZARIBAGH', 'HISAR', 'HOJAI', 'HOSHIARPUR', 'HOWRAH', 'HYDERABAD', 'IDUKKI', 'IMPHAL EAST', 'IMPHAL WEST', 'JAGATSinghapur', 'JAGTIAL', 'JAISALMER', 'JAJAPUR', 'JALANDHAR', 'JALORE', 'JALPAIGURI', 'JAMMU', 'JAMTARA', 'JAMUI', 'JANGOAN', 'JANJGIR-CHAMPA', 'JASHPUR', 'JAUNPUR', 'JAYASHANKAR', 'JEHANABAD', 'JHABUA', 'JHAJJAR', 'JHARGRAM', 'JHARSUGUDA', 'JIND', 'JOGULAMBA', 'JORHAT', 'KABIRDHAM', 'KACHCHH', 'KAIMUR (BHABUA)', 'KAITHAL', 'KALAHANDI', 'KALIMPONG', 'KALLAKURICHI', 'KAMAREDDY', 'KAMLE', 'KAMRUP', 'KAMRUP METRO', 'KANCHIPURAM', 'KANDHAMAL', 'KANGRA', 'KANKER', 'KANNAUJ', 'KANNIYAKUMARI', 'KANNUR', 'KANPUR DEHAT', 'KANPUR NAGAR',

'KAPURTHALA', 'KARAIKAL', 'KARALI', 'KARBI ANGLONG', 'KARGIL',
'KARIMGANJ', 'KARNAL', 'KARUR', 'KASARAGOD', 'KASGANJ', 'KATHUA',
'KATIHAR', 'KATNI', 'KAUSHAMBI', 'KENDRAPARA', 'KENDUJHAR',
'KHAGARIA', 'KHAMMAM', 'KHANDWA', 'KHEDA', 'KHERI', 'KHORDHA',
'KHOWAI', 'KHUNTI', 'KINNAUR', 'KIPHIRE', 'KISHANGANJ', 'KISHTWAR',
'KODAGU', 'KODERMA', 'KOHIMA', 'KOKRAJHAR', 'KOLAR', 'KOLASIB',
'KOLLAM', 'KOMARAM BHEEM ASIFABAD', 'KONDAGAON', 'KORAPUT', 'KORBA',
'KOREA', 'KOTTAYAM', 'KOZHIKODE', 'KRA DAADI', 'KRISHNAGIRI',
'KULGAM', 'KULLU', 'KUPWARA', 'KURUKSHETRA', 'KURUNG KUMEY', 'KUSHI
NAGAR', 'LAHUL AND SPITI', 'LAKHIMPUR', 'LAKHISARAI', 'LATEHAR',
'LAWNGTLAI', 'LEH LADAKH', 'LEPARADA', 'LOHARDAGA', 'LOHIT',
'LONGDING', 'LONGLENG', 'LOWER DIBANG VALLEY', 'LOWER SIANG', 'LOWER
SUBANSIRI', 'LUCKNOW', 'LUDHIANA', 'LUNGLEI', 'MADHEPURA',
'MADHUBANI', 'MADURAI', 'MAHABUBABAD', 'MAHARAJGANJ', 'MAHASAMUND',
'MAHE', 'MAHENDRAGARH', 'MAHESANA', 'MAHISAGAR', 'MAINPURI', 'MAJULI\`
r\n', 'MALAPPURAM', 'MALDAH', 'MALKANGIRI', 'MAMIT', 'MANCHERIAL',
'MANDI', 'MANDLA', 'MANDYA', 'MANSA', 'MARIGAON', 'MATHURA', 'MAU',
'MAYURBHANJ', 'MEDCHAL MALKAJGIRI', 'MEERUT', 'MEWAT', 'MIRZAPUR',
'MOGA', 'MOKOKCHUNG', 'MON', 'MORADABAD', 'MORBI', 'MORENA',
'MUKTSAR', 'MULUGU', 'MUMBAI', 'MUMBAI SUBURBAN', 'MUNGELI', 'MUNGER',
'MUZAFFARNAGAR', 'MUZAFFARPUR', 'NABARANGPUR', 'NAGAON',
'NAGAPATTINAM', 'NAGARKURNOOL', 'NAINITAL', 'NALANDA', 'NALBARI',
'NALGONDA', 'NAMAKKAL', 'NAMSAI', 'NARAYANAPET', 'NARAYANPUR',
'NARMADA', 'NAVSARI', 'NAWADA', 'NAYAGARH', 'NICOBARS', 'NIRMAL',
'NIWARI', 'NORTH AND MIDDLE ANDAMAN', 'NORTH DISTRICT', 'NORTH GARO
HILLS', 'NORTH GOA', 'NORTH TRIPURA', 'NUAPADA', 'PAKKE KESSANG',
'PAKUR', 'PALAKKAD', 'PALAMU', 'PALGHAR', 'PALI', 'PALWAL', 'PANCH
MAHALS', 'PANCHKULA', 'PANIPAT', 'PAPUM PARE', 'PARAGANAS SOUTH',
'PASCHIM BARDHAMAN\r\n', 'PASHCHIM CHAMPARAN', 'PATAN',
'PATHANAMTHITTA', 'PATHANKOT', 'PATIALA', 'PATNA', 'PAURI GARHWAL',
'PEDDAPALLI', 'PERAMBALUR', 'PEREN', 'PHEK', 'PILIBHIT',
'PITHORAGARH', 'PONDICHERY', 'POONCH', 'PORBANDAR', 'PRATAPGARH',
'PUDUKKOTTAI', 'PULWAMA', 'PURBI CHAMPARAN', 'PURI', 'PURNIA',
'PURULIA', 'RAE BARELI', 'RAIGAD', 'RAIGARH', 'RAIPUR', 'RAJANNA',
'RAJAURI', 'RAJNANDGAON', 'RAJSAMAND', 'RAMANAGARA', 'RAMANATHAPURAM',
'RAMBAN', 'RAMGARH', 'RAMPUR', 'RANCHI', 'RANGAREDDI', 'RANIPET',
'RATNAGIRI', 'RAYAGADA', 'REASI', 'REWARI', 'RI BHOI', 'ROHTAK',
'ROHTAS', 'RUDRA PRAYAG', 'RUPNAGAR', 'S NAGAR', 'SABAR KANTHA',
'SAHARANPUR', 'SAHARSA', 'SAHEBGANJ', 'SAIHA', 'SALEM', 'SAMASTIPUR',
'SAMBA', 'SAMBALPUR', 'SAMBHAL', 'SANGAREDDY', 'SANGRUR', 'SANT KABEER
NAGAR', 'SANT RAVIDAS NAGAR', 'SARAIKELA KHARSAWAN', 'SARAN', 'SAWAI
MADHOPUR', 'SENAPATI', 'SEPAHIJALA', 'SERCHHIP', 'SHAHDOL', 'SHAHID
BHAGAT SINGH NAGAR', 'SHAHJAHANPUR', 'SHAMLI', 'SHEIKHPURA',
'SHEOHAR', 'SHEOPUR', 'SHI YOMI', 'SHIMLA', 'SHIVAMOGGA', 'SHOPIAN',
'SHRAVASTI', 'SIANG', 'SIDDHARTH NAGAR', 'SIDDIPET', 'SIMDEGA',
'SINDHUDURG', 'SINGRAULI', 'SIRMAUR', 'SIROHI', 'SIRSA', 'SITAMARHI',
'SITAPUR', 'SIVAGANGA', 'SIVASAGAR', 'SIWAN', 'SOLAN', 'SONBHADRA',
'SONEPUR', 'SONIPAT', 'SONITPUR', 'SOUTH ANDAMANS', 'SOUTH DISTRICT',
'SOUTH GARO HILLS', 'SOUTH GOA', 'SOUTH SALMARA MANCACHAR\r\n', 'SOUTH

TRIPURA', 'SOUTH WEST GARO HILLS', 'SOUTH WEST KHASI HILLS', 'SPSR NELLORE', 'SRIKAKULAM', 'SRINAGAR', 'SUKMA', 'SULTANPUR', 'SUNDARGARH', 'SUPAUL', 'SURAJPUR', 'SURAT', 'SURGUJA', 'SURYAPET', 'TAMENGLONG', 'TAPI', 'TARN TARAN', 'TAWANG', 'TEHRI GARHWAL', 'TENKASI', 'THANE', 'THANJAVUR', 'THE NILGIRIS', 'THENI', 'THIRUVALLUR', 'THIRUVANANTHAPURAM', 'THIRUVARUR', 'THOUBAL', 'THRISSUR', 'TINSUKIA', 'TIRAP', 'TIRUCHIRAPPALLI', 'TIRUNELVELI', 'TIRUPATHUR', 'TIRUPPUR', 'TIRUVANNAMALAI', 'TONK', 'TUENSANG', 'TUMAKURU', 'TUTICORIN', 'UDAIPUR', 'UDALGURI', 'UDAM SINGH NAGAR', 'UDHAMPUR', 'UDUPI', 'UKHRUL', 'UMARIA', 'UNA', 'UNAKOTI', 'UNNAO', 'UPPER SIANG', 'UPPER SUBANSIRI', 'UTTAR KASHI', 'UTTARA KANNADA', 'VAISHALI', 'VALSAD', 'VARANASI', 'VELLORE', 'VIKARABAD', 'VILLUPURAM', 'VIRUDHUNAGAR', 'VISAKHAPATANAM', 'WANAPARTHY', 'WAYANAD', 'WEST DISTRICT', 'WEST GARO HILLS', 'WEST JAINTIA HILLS', 'WEST KAMENG', 'WEST KARBI ANGLONG\r\n', 'WEST KHASI HILLS', 'WEST SIANG', 'WEST SINGHBHUM', 'WEST TRIPURA', 'WOKHA', 'YADADRI', 'YADAGIRI', 'YAMUNANAGAR', 'YANAM', 'ZUNHEBOTO']

Cluster 2 characteristics:
Number of districts: 14
Top 5 crops by production:
Crop
Coconut 2.368791e+10
Rice 3.306836e+08
Potato 1.841287e+08
Jute 1.586406e+08
Sugarcane 1.264099e+08
Name: Production, dtype: float64

Districts in this cluster:
['BANKURA', 'BIRBHUM', 'COOCHBEHAR', 'DINAJPUR UTTAR', 'EAST GODAVARI', 'HOOGHLY', 'MEDINIPUR EAST', 'MEDINIPUR WEST', 'MURSHIDABAD', 'NADIA', 'PARAGANAS NORTH', 'PURBA BARDHAMAN', 'VIZIANAGARAM', 'WEST GODAVARI']

Cluster 3 characteristics:
Number of districts: 12
Top 5 crops by production:
Crop
Wheat 91046661.0
Bajra 63393277.0
Rapeseed & Mustard 36981566.0
Cotton(lint) 23697536.0
Guar seed 21664367.0
Name: Production, dtype: float64

Districts in this cluster:
['ALWAR', 'BANAS KANTHA', 'BARMER', 'BIKANER', 'CHURU', 'GANGANAGAR', 'HANUMANGARH', 'JAIPUR', 'JHUNJHUNU', 'JODHPUR', 'NAGAU', 'SIKAR']

Cluster 4 characteristics:
Number of districts: 2
Top 5 crops by production:
Crop
Coconut 1.038339e+10
Sugarcane 2.812631e+07
Rice 1.033128e+07
Maize 6.048382e+06
Ragi 5.366618e+06
Name: Production, dtype: float64

Districts in this cluster:
['HASSAN', 'MYSURU']

=== Cluster Summary ===

Number of districts in each cluster:
1 581
0 98
2 14
3 12
4 2
Name: Cluster, dtype: int64

Dominant crops by cluster:

Cluster	Dominant_Crop	
0	Wheat	35
	Sugarcane	26
	Coconut	13
	Cotton(lint)	12
	Rice	5
	Soyabean	3
	Groundnut	1
	Maize	1
	Onion	1
	Potato	1
1	Rice	201
	Coconut	122
	Wheat	110
	Sugarcane	71
	Maize	33
	Potato	13
	Cotton(lint)	10
	Jute	5
	Banana	3
	Bajra	2
	Ginger	2
	Groundnut	2
	Rapeseed &Mustard	2
	Arecanut	1

	Barley	1
	Dry chillies	1
	Gram	1
	Linseed	1
2	Coconut	13
	Jute	1
3	Bajra	6
	Wheat	4
	Guar seed	1
	Potato	1
4	Coconut	2
Name: Dominant_Crop, dtype: int64		

What the Scatter Plot above means:

Clusters and Colors:

- Each dot represents a district
- The colors represent different clusters (from dark purple to yellow)
- Districts with the same color have similar crop production patterns

Position Meaning:

- The x-axis (First Principal Component) and y-axis (Second Principal Component) represent combined patterns of crop production
- Districts that are close together on the plot have similar crop production patterns
- Districts that are far apart have very different patterns

Key Observations:

- There's a dense blue cluster on the left side (-2, 0), suggesting many districts share a common agricultural pattern
- A scattered purple cluster spreads across the right side, indicating districts with more unique patterns
- Some outlier districts appear at the top of the plot (around y=15), suggesting very distinctive production patterns
- The spread along both axes shows significant variation in how districts produce crops

Practical Interpretation:

- The large blue cluster likely represents districts with "typical" or common crop production patterns
- The scattered points on the right and top might be districts specializing in specific crops or having unique agricultural profiles
- The gradual transition between clusters suggests some districts have hybrid production patterns

Now that we've covered what the scatter plot means, let's look at the geographical regions the clusters correspond to and majority of the crop

```
print("\nCluster assignments:")
print(cluster_APY.head(5))
```

```
print("\nDominant crops:")
print(dominant_crops.head(5))
```

Cluster assignments:

	District	Cluster	Dominant_Crop
6	ADILABAD	0	Cotton(lint)
70	AGAR MALWA	1	Wheat
95	AGRA	1	Potato
111	AHMADABAD	1	Cotton(lint)
161	AHMEDNAGAR	0	Sugarcane

Dominant crops:

	District	Crop	Production
6	ADILABAD	Cotton(lint)	9684901.0
70	AGAR MALWA	Wheat	1474013.0
95	AGRA	Potato	27685472.0
111	AHMADABAD	Cotton(lint)	6020981.0
161	AHMEDNAGAR	Sugarcane	151667127.0

```
geo_data = india_dt.merge(
    cluster_APY,
    left_on='DISTRICT',
    right_on='District',
    how='left'
)
```

Keep it as a GeoDataFrame

```
geo_data = gpd.GeoDataFrame(geo_data)
```

Calculate centroids

```
geo_data['longitude'] = geo_data.geometry.centroid.x
geo_data['latitude'] = geo_data.geometry.centroid.y
```

1. Clean and prepare the data

```
geo_data['DISTRICT_CLEAN'] =
geo_data['DISTRICT'].str.strip().str.upper()
cluster_APY_clean = cluster_APY.copy()
cluster_APY_clean['District'] =
cluster_APY_clean['District'].str.strip().str.upper()
```

2. Perform merge with clean column names

```
geo_data_new = geo_data.merge(
```

```

cluster_APY_clean,
left_on='DISTRICT_CLEAN',
right_on='District',
how='left',
suffixes=(None, '_y') # This will prevent duplicate columns
).drop(columns=[col for col in geo_data.columns if
col.endswith('_y')])

# 3. Convert back to GeoDataFrame
geo_data_new = gpd.GeoDataFrame(geo_data_new, geometry='geometry')

def clean_and_prepare_geo_data(india_dt, cluster_APY):
    """
    Clean and prepare the geographical data with proper column
    handling
    """
    # Create a copy of the dataframes
    geo_data = india_dt.copy()
    cluster_data = cluster_APY.copy()

    # First, let's see what columns we actually have
    print("\nOriginal columns in geo_data:")
    print(geo_data.columns.tolist())
    print("\nOriginal columns in cluster_data:")
    print(cluster_data.columns.tolist())

    # Clean district names
    geo_data['DISTRICT_CLEAN'] =
geo_data['DISTRICT'].str.strip().str.upper()
    cluster_data['District'] =
cluster_data['District'].str.strip().str.upper()

    # Perform the merge
    geo_data_merged = geo_data.merge(
        cluster_data,
        left_on='DISTRICT_CLEAN',
        right_on='District',
        how='left'
    )

    # Keep it as a GeoDataFrame
    geo_data_new = gpd.GeoDataFrame(geo_data_merged,
    geometry='geometry')

    print("\nColumns after merge:")
    print(geo_data_new.columns.tolist())

    # Calculate centroids if not already present
    if 'longitude' not in geo_data_new.columns:
        geo_data_new['longitude'] = geo_data_new.geometry.centroid.x

```

```

        geo_data_new['latitude'] = geo_data_new.geometry.centroid.y

# Keep only the columns we need with their original names
columns_to_keep = [
    'DST_ID',
    'DISTRICT',
    'STATE',
    'geometry',
    'Cluster', # Use original column name
    'Dominant_Crop',
    'longitude',
    'latitude'
]

# Filter columns that actually exist
existing_columns = [col for col in columns_to_keep if col in
geo_data_new.columns]

geo_data_clean = geo_data_new[existing_columns]

print("\nData Summary:")
print(f"Total districts: {len(geo_data_clean)}")
print(f"Districts with cluster assignments:
{geo_data_clean['Cluster'].notna().sum()}")
print("\nCluster distribution:")
print(geo_data_clean['Cluster'].value_counts(dropna=False))

return geo_data_clean

def plot_agricultural_clusters(geo_data):
    """
    Plot agricultural clusters with proper handling of missing values
    """
    fig, ax = plt.subplots(figsize=(15, 10))

    # Create the plot
    geo_data.plot(
        ax=ax,
        column='Cluster',
        categorical=True,
        legend=True,
        cmap='Set3',
        missing_kwds={'color': 'lightgrey'},
        edgecolor='black',
        linewidth=0.5,
        legend_kwds={
            'title': 'Agricultural\nCluster',
            'bbox_to_anchor': (1.3, 1)
        }
    )

```

```

)

# Customize the plot
ax.set_title('Agricultural Clusters by District', pad=20)
ax.axis('off')

# Add text explaining missing data
plt.figtext(
    0.99, 0.02,
    'Grey areas represent districts with no cluster data',
    ha='right', fontsize=8, style='italic'
)

plt.tight_layout()
plt.show()

# Try the complete analysis with debugging information
try:
    # Clean and prepare the data
    geo_data_clean = clean_and_prepare_geo_data(india_dt, cluster_APY)

    # Create the visualization
    plot_agricultural_clusters(geo_data_clean)

except Exception as e:
    print(f"\nError in execution: {str(e)}")
    import traceback
    traceback.print_exc()

```

Original columns in geo_data:

```
['DST_ID', 'DISTRICT', 'STATE', 'geometry']
```

Original columns in cluster_data:

```
['District', 'Cluster', 'Dominant_Crop']
```

Columns after merge:

```
['DST_ID', 'DISTRICT', 'STATE', 'geometry', 'DISTRICT_CLEAN',
'District', 'Cluster', 'Dominant_Crop']
```

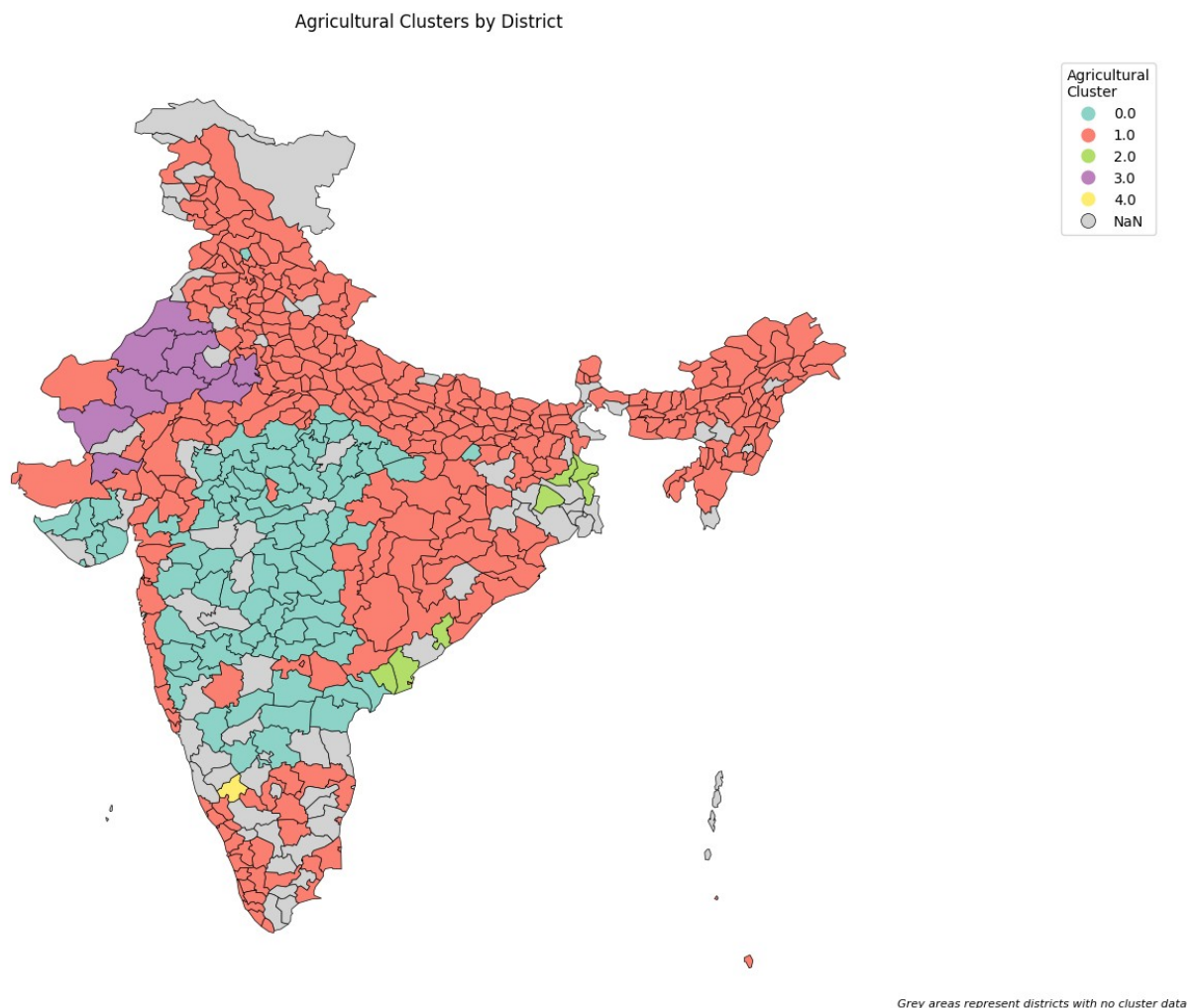
Data Summary:

```
Total districts: 482
Districts with cluster assignments: 401
```

Cluster distribution:

```
1.0    306
NaN     81
0.0     77
3.0     10
2.0      7
```

4.0 1
Name: Cluster, dtype: int64



create a detailed_district_report function that can retrieve a lot of information about a district's production

```
def create_detailed_district_report(district_name, df):  
    """Create a detailed report for a specific district"""  
    district_data = df[df['District'] == district_name]  
    yearly_production = district_data.groupby(['Crop', 'Crop_Year'])  
    ['Production'].sum().reset_index()  
  
    average_yearly_production = yearly_production.groupby('Crop')  
    ['Production'].mean()\br/>        .sort_values(ascending=False)\br/>        .head()  
  
    print(f"\n=== Detailed Report for {district_name} ===")
```

```

print("\nTop 5 crops by production:")
print(average_yearly_production)

print("\nSeasonal distribution:")
print(district_data['Season'].value_counts())

top_crop = district_data.groupby('Crop')
['Production'].sum().idxmax()
yearly_trend = district_data[district_data['Crop'] ==
top_crop].groupby('Crop_Year')['Production'].sum()

plt.figure(figsize=(10, 6))
yearly_trend.plot(kind='line')
plt.title(f'Production Trend of {top_crop} in {district_name}')
plt.xlabel('Year')
plt.ylabel('Production')
plt.show()

```

Using the **create_detailed_district_report** function can retrieve a lot of information about a district's production of the crops since 1961 on average.

```

#only enter the district's name in capital letters
create_detailed_district_report('GUNTUR', APY)

```

```

=== Detailed Report for GUNTUR ===

```

```

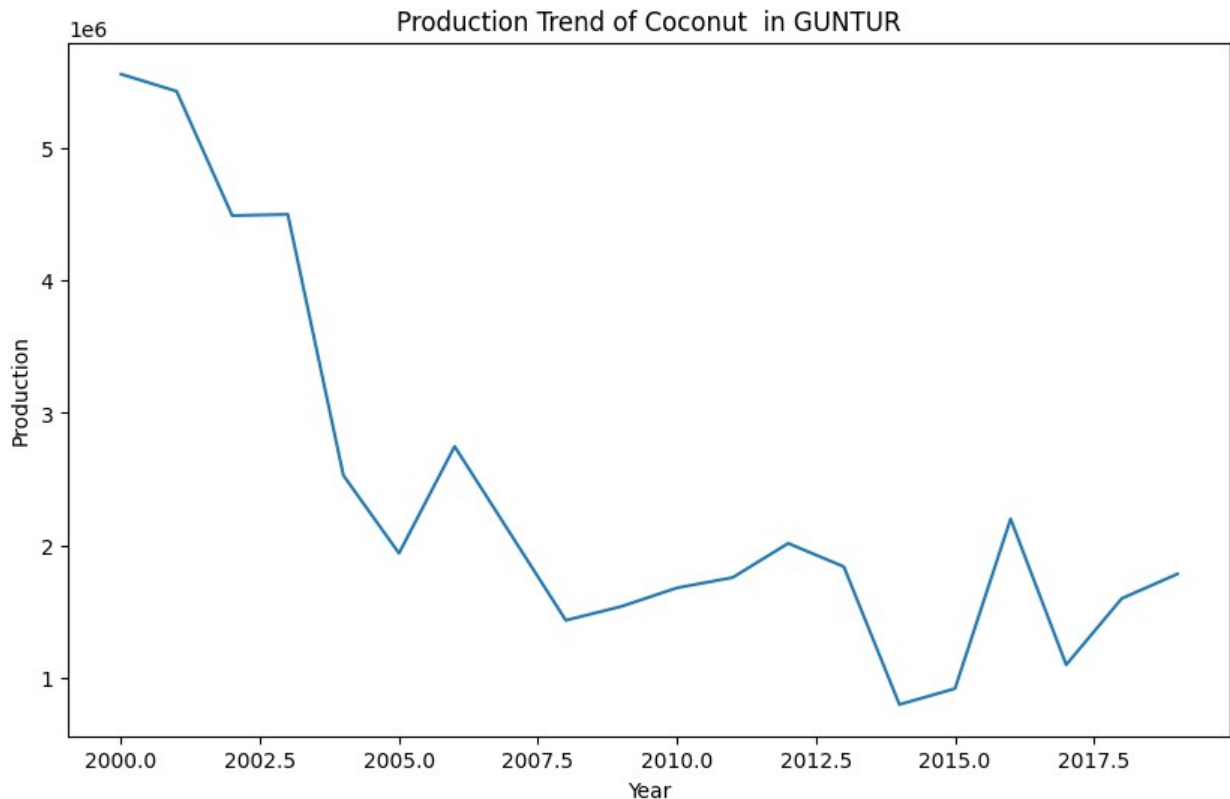
Top 5 crops by production:
Crop
Coconut      2.413576e+06
Rice          9.199871e+05
Cotton(lint)  5.179713e+05
Maize         3.774262e+05
Dry chillies  2.739435e+05
Name: Production, dtype: float64

```

```

Seasonal distribution:
Rabi          350
Kharif         348
Whole Year     121
Name: Season, dtype: int64

```

Implementing Time Analysis and forecasting now:

- First create time window, prepare climate features and the build the lstm model

```
def prepare_climate_features(rainfall_df, temp_df):
    """
    Create relevant climate features from rainfall and temperature
    data
    """
    try:
        # Create copies to avoid modifying originals
        rainfall_features = rainfall_df.copy()
        temp_features = temp_df.copy()

        # Create a feature DataFrame with YEAR as index
        features = pd.DataFrame()
        features['YEAR'] = rainfall_features['YEAR']

        # Add existing seasonal rainfall if available
        if 'Jun-Sep' in rainfall_features.columns:
            features['monsoon_rainfall'] = rainfall_features['Jun-
Sep']

            features['pre_monsoon'] = rainfall_features['Mar-May']
            features['post_monsoon'] = rainfall_features['Oct-Dec']
        else:
            # Calculate seasonal rainfall from monthly data
```

```

        features['monsoon_rainfall'] = rainfall_features[['JUN',
'JUL', 'AUG', 'SEP']].sum(axis=1)
        features['pre_monsoon'] = rainfall_features[['MAR', 'APR',
'MAY']].sum(axis=1)
        features['post_monsoon'] = rainfall_features[['OCT',
'NOV', 'DEC']].sum(axis=1)

        # Add temperature features
        features['annual_temp'] = temp_features['ANNUAL']
        features['temp_variation'] = temp_features['JUN-SEP'] -
temp_features['JAN-FEB']

        print("\nFeatures created successfully:")
        print(features.columns.tolist())
        print("\nSample of features:")
        print(features.head())

        return features

    except Exception as e:
        print(f"Error in prepare_climate_features: {str(e)}")
        print("\nAvailable columns in rainfall_df:",
rainfall_df.columns.tolist())
        print("\nAvailable columns in temp_df:",
temp_df.columns.tolist())
        raise

def create_climate_production_dataset(APY_df, rainfall_df, temp_df,
crop_name, district=None):
    """
    Create combined dataset of production and climate data with better
    error handling
    """
    try:
        # Filter for specific crop and district
        if district:
            crop_data = APY_df[(APY_df['Crop'] == crop_name) &
                                (APY_df['District'] == district)].copy()
        else:
            crop_data = APY_df[APY_df['Crop'] == crop_name].copy()

        print(f"\nFound {len(crop_data)} records for {crop_name} in
{district if district else 'all districts'}")

        # Group by year
        yearly_production = crop_data.groupby('Crop_Year')
        ['Production'].sum().reset_index()
        yearly_production.columns = ['YEAR', 'Production']

```

```

        # Prepare climate features
        climate_features = prepare_climate_features(rainfall_df,
temp_df)

        # Merge production and climate data
        combined_data = pd.merge(yearly_production, climate_features,
on='YEAR', how='inner')
        combined_data = combined_data.sort_values('YEAR')

        print("\nCombined dataset shape:", combined_data.shape)
        print("\nColumns in combined dataset:",
combined_data.columns.tolist())
        print("\nFirst few rows of combined dataset:")
        print(combined_data.head())

        return combined_data

    except Exception as e:
        print(f"\nError in create_climate_production_dataset:
{str(e)}")
        print("\nDebug information:")
        print("APY_df columns:", APY_df.columns.tolist())
        print("Rainfall columns:", rainfall_df.columns.tolist())
        print("Temperature columns:", temp_df.columns.tolist())
        raise

def create_time_windows(data, window_size):
    """
    Create time windows for sequence prediction
    """
    X, y = [], []
    for i in range(len(data) - window_size):
        X.append(data[i:(i + window_size)])
        y.append(data[i + window_size])
    return np.array(X), np.array(y)

def build_lstm_model(input_shape, dropout_rate=0.2):
    """
    Build LSTM model for time series forecasting
    """
    model = Sequential([
        LSTM(64, activation='relu', input_shape=input_shape,
return_sequences=True),
        Dropout(dropout_rate),
        LSTM(32, activation='relu'),
        Dropout(dropout_rate),
        Dense(16, activation='relu'),
        Dense(1)
    ])

```

```
model.compile(optimizer='adam', loss='mse', metrics=['mae'])
return model
```

Now Analyze seasonality and create climate production dataset by:

1. Filtering production data for specific crop/district
2. Then aggregating production by year
3. After which we merge with climate features and return time-ordered dataset with production and climate variables

```
def analyze_seasonality(data, period=12):
    """
    Analyze seasonal patterns in time series data with error handling
    """
    # Check if we have enough data
    if len(data) < period * 2:
        print(f"Warning: Not enough data for seasonality analysis.
        Need at least {period * 2} points, but got {len(data)}")
        return None, None

    try:
        # Calculate lag correlation
        result = stats.pearsonr(data[:-period], data[period:])
        lag_correlation = result[0]

        # Calculate seasonal indices
        seasonal_means = []
        for i in range(period):
            values = data[i::period]
            if len(values) > 0: # Check if we have values for this
season
                seasonal_means.append(np.mean(values))
            else:
                seasonal_means.append(np.nan)

        if any(not np.isnan(x) for x in seasonal_means): # If we have
any valid means
            seasonal_index = np.array(seasonal_means) /
np.nanmean(seasonal_means)
        else:
            seasonal_index = np.array([np.nan] * period)

        return lag_correlation, seasonal_index

    except Exception as e:
        print(f"Error in seasonality analysis: {str(e)}")
        return None, None

#Now we create a sequence data using time windows, obviously splitting
the data into a 80-20 split and training the lstm model
```

```

def train_production_forecast_model(combined_data, feature_columns,
target_column='Production',
                                window_size=5, train_split=0.8):
    """
    Train LSTM model for production forecasting
    """
    # Scale data
    scaler = MinMaxScaler()
    scaled_data = scaler.fit_transform(combined_data[feature_columns +
[target_column]])

    # Create sequences
    X, y = create_time_windows(scaled_data, window_size)

    # Split into train and test
    train_size = int(len(X) * train_split)
    X_train, X_test = X[:train_size], X[train_size:]
    y_train, y_test = y[:train_size], y[train_size:]

    # Build and train model
    model = build_lstm_model((window_size, len(feature_columns) + 1))
    history = model.fit(
        X_train, y_train,
        epochs=100,
        batch_size=32,
        validation_split=0.2,
        verbose=1
    )

    return model, scaler, history

```

1. Calculate correlations between each climate variable and production
2. Create scatter plots showing relationships
3. Return correlation coefficients

```

def forecast_future_production(model, scaler, last_sequence,
n_steps=10):
    """
    Generate future production forecasts
    """
    current_sequence = last_sequence.copy()
    forecasts = []

    for _ in range(n_steps):
        prediction = model.predict(current_sequence.reshape(1,
*current_sequence.shape), verbose=0)
        forecasts.append(prediction[0, 0])

    # Update sequence

```

```

        current_sequence = np.roll(current_sequence, -1, axis=0)
        current_sequence[-1] = prediction

    # Inverse transform predictions
    forecast_values = scaler.inverse_transform(
        np.column_stack([np.zeros((len(forecasts),
scaler.scale_.shape[0]-1)), forecasts])
    )[:, -1]

    return forecast_values

def plot_forecast_results(combined_data, forecast, crop_name,
district=None):
    """
    Plot historical data and forecasts
    """
    plt.figure(figsize=(15, 7))

    # Plot historical data
    plt.plot(combined_data['YEAR'], combined_data['Production'],
             label='Historical', linewidth=2)

    # Generate future years matching forecast length
    future_years = np.arange(
        combined_data['YEAR'].max() + 1,
        combined_data['YEAR'].max() + len(forecast) + 1
    )

    # Plot forecast
    plt.plot(future_years, forecast, '--', label='Forecast',
linewidth=2)

    # Add confidence intervals (using historical standard deviation)
    std_dev = combined_data['Production'].std()
    plt.fill_between(future_years,
                     forecast - std_dev,
                     forecast + std_dev,
                     alpha=0.2,
                     label='Confidence Interval')

    title = f'{crop_name} Production Forecast'
    if district:
        title += f' for {district}'
    plt.title(title)
    plt.xlabel('Year')
    plt.ylabel('Production')
    plt.grid(True, alpha=0.3)
    plt.legend()

```

```

# Add value labels
# for year, value in zip(future_years, forecast):
#     plt.text(year, value, f'{value:.0f}',
#             horizontalalignment='center',
#             verticalalignment='bottom')

plt.tight_layout()
plt.show()

# Print forecast values
print("\nForecast Values:")
print("Year\t\t\tPredicted Production")
print("-" * 35)
for year, value in zip(future_years, forecast):
    print(f"{year}\t\t\t{value:,.2f}")

```

As we can see in the previous few code blocks, the code format allows for data integration of climate dataset with the crop production and figure out their correlation, make an

```

def analyze_climate_impact(combined_data):
    """
    Analyze relationship between climate variables and production with
    improved error handling
    """
    try:
        # Define climate columns that should exist in the data
        climate_cols = ['monsoon_rainfall', 'pre_monsoon',
                        'post_monsoon',
                        'annual_temp', 'temp_variation']

        # Verify which columns actually exist
        available_cols = [col for col in climate_cols if col in
                           combined_data.columns]

        print("\nAnalyzing climate impact for columns:",
              available_cols)

        correlations = {}

        # Calculate correlations for available columns
        for col in available_cols:
            # Check for NaN values
            if combined_data[col].isnull().any() or
                combined_data['Production'].isnull().any():
                print(f"Warning: NaN values found in {col} or
Production")
                continue

```

```

        correlation = stats.pearsonr(combined_data[col],
combined_data['Production'])
        correlations[col] = correlation[0]
        print(f"Correlation for {col}: {correlation[0]:.3f}")

    # Plot relationships
    if correlations:
        n_cols = len(available_cols)
        fig_width = min(15, 5 * n_cols)
        fig = plt.figure(figsize=(fig_width, 5))

        for i, col in enumerate(available_cols, 1):
            plt.subplot(1, n_cols, i)
            plt.scatter(combined_data[col],
combined_data['Production'])
            plt.xlabel(col)
            plt.ylabel('Production')
            plt.title(f'Correlation: {correlations[col]:.2f}')

        plt.tight_layout()
        plt.show()

    return correlations

except Exception as e:
    print(f"\nError in analyze_climate_impact: {str(e)}")
    print("\nDebug information:")
    print("Available columns:", combined_data.columns.tolist())
    print("\nSample of data:")
    print(combined_data.head())
    raise

```

As we can see above, the code format allows for data integration of climate dataset with the crop production and figure out their correlation

```

def plot_forecast_results(historical_data, forecast, crop_name,
district):
    """
    Create a comprehensive visualization of the forecast results
    """
    plt.figure(figsize=(15, 10))

    # Plot historical data
    plt.subplot(2, 1, 1)
    plt.plot(historical_data.index, historical_data['Production'],
            label='Historical Production', marker='o')

```



```

plt.title(f'Historical {crop_name} Production in {district}')
plt.xlabel('Year')
plt.ylabel('Production')
plt.grid(True)
plt.legend()

# Plot forecast with confidence intervals
plt.subplot(2, 1, 2)
last_year = historical_data.index[-1]
forecast_years = range(last_year + 1, last_year + len(forecast) +
1)

plt.plot(historical_data.index[-5:],
historical_data['Production'].iloc[-5:],
        label='Recent Production', marker='o')
plt.plot(forecast_years, forecast, label='Forecast', marker='s',
color='red')
plt.title(f'{crop_name} Production Forecast for {district}')
plt.xlabel('Year')
plt.ylabel('Production')
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.show()

# Print forecast values
print("\nProduction Forecast:")
for year, value in zip(forecast_years, forecast):
    print(f"Year {year}: {value:,.0f}")

# Print correlation analysis
print("\nClimate Correlation Analysis:")
for col in historical_data.columns:
    if col != 'Production':
        corr =
historical_data[col].corr(historical_data['Production'])
        print(f"{col}: {corr:.3f}")

def main_analysis2(APY_df, rainfall_df, temp_df, crop_name,
district=None, n_years=10):
    try:
        print(f"\nAnalyzing {crop_name} production in {district}")
        print("=" * 50)

        # Create dataset
        combined_data = create_climate_production_dataset(
            APY_df, rainfall_df, temp_df, crop_name, district
        )

```

```

    # Analyze climate impact
    climate_correlations = analyze_climate_impact(combined_data)

    # Train model and generate forecast
    feature_columns = ['monsoon_rainfall', 'pre_monsoon',
                       'post_monsoon',
                       'annual_temp', 'temp_variation']
    model, scaler, history = train_production_forecast_model(
        combined_data, feature_columns
    )

    # Generate forecast
    last_sequence = combined_data[feature_columns +
    ['Production']].iloc[-5:]
    last_sequence_scaled = scaler.transform(last_sequence)
    forecast = forecast_future_production(model, scaler,
    last_sequence_scaled, n_steps=n_years)

    # Plot results
    plot_forecast_results(combined_data, forecast, crop_name,
    district)

    print("\nModel Training Performance:")
    print(f"Final loss: {history.history['loss'][-1]:.4f}")
    print(f"Final MAE: {history.history['mae'][-1]:.4f}")

    return {
        'model': model,
        'forecast': forecast,
        'climate_impact': climate_correlations,
        'data': combined_data
    }

except Exception as e:
    print(f"\nError in analysis: {str(e)}")
    raise

# Try running the analysis with the modified code
try:
    results = main_analysis2(APY, rainfall, temp, crop_name='Rice',
    district='WARANGAL')
except Exception as e:
    print(f"\nAnalysis failed: {str(e)}")
    print("\nTrying to debug the data:")
    print("\nSample of APY data for Rice in WARANGAL:")
    print(APY[(APY['Crop'] == 'Rice') & (APY['District'] ==
    'WARANGAL')][['Crop_Year', 'Production']].head())
    print("\nSample of rainfall data:")
    print(rainfall.head())

```

```
print("\nSample of temperature data:")
print(temp.head())
```

Analyzing Rice production in WARANGAL

=====

Found 46 records for Rice in WARANGAL

Features created successfully:

```
['YEAR', 'monsoon_rainfall', 'pre_monsoon', 'post_monsoon',
'annual_temp', 'temp_variation']
```

Sample of features:

	YEAR	monsoon_rainfall	pre_monsoon	post_monsoon	annual_temp \
0	1961	1052.0	132.1	154.2	24.00
1	1962	906.6	130.4	126.7	24.04
2	1963	908.2	153.1	142.9	24.15
3	1964	997.0	111.1	103.5	24.10
4	1965	708.5	124.6	74.4	24.07

	temp_variation
0	8.08
1	8.23
2	8.14
3	8.07
4	7.86

Combined dataset shape: (22, 7)

Columns in combined dataset: ['YEAR', 'Production', 'monsoon_rainfall', 'pre_monsoon', 'post_monsoon', 'annual_temp', 'temp_variation']

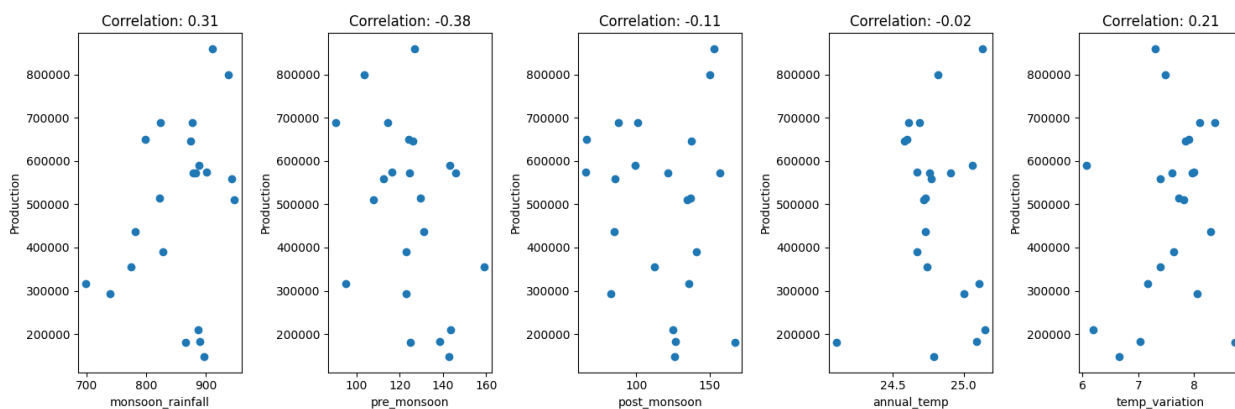
First few rows of combined dataset:

	YEAR	Production	monsoon_rainfall	pre_monsoon	post_monsoon	annual_temp \
0	1997	179900.0	866.5	124.8	167.0	24.10
1	1998	572800.0	882.6	124.6	157.0	24.76
2	1999	389493.0	827.9	122.9	141.0	24.67
3	2000	650024.0	798.1	124.3	66.4	24.60
4	2001	515035.0	822.4	129.7	137.1	24.73

	temp_variation
0	8.73

1	7.97
2	7.63
3	7.90
4	7.72

Analyzing climate impact for columns: ['monsoon_rainfall', 'pre_monsoon', 'post_monsoon', 'annual_temp', 'temp_variation']
 Correlation for monsoon_rainfall: 0.305
 Correlation for pre_monsoon: -0.379
 Correlation for post_monsoon: -0.111
 Correlation for annual_temp: -0.017
 Correlation for temp_variation: 0.214



Epoch 1/100

```
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/
site-packages/keras/src/layers/rnn/rnn.py:200: UserWarning: Do not
pass an `input_shape`/`input_dim` argument to a layer. When using
Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
  super().__init__(**kwargs)
```

1/1 1s 1s/step - loss: 0.3948 - mae: 0.5667 -
 val_loss: 0.3922 - val_mae: 0.5661

Epoch 2/100

1/1 0s 35ms/step - loss: 0.3781 - mae: 0.5521 -
 val_loss: 0.3824 - val_mae: 0.5579

Epoch 3/100

1/1 0s 36ms/step - loss: 0.3701 - mae: 0.5456 -
 val_loss: 0.3730 - val_mae: 0.5504

Epoch 4/100

1/1 0s 36ms/step - loss: 0.3619 - mae: 0.5386 -
 val_loss: 0.3647 - val_mae: 0.5435

Epoch 5/100

1/1 0s 35ms/step - loss: 0.3519 - mae: 0.5305 -
 val_loss: 0.3560 - val_mae: 0.5362

Epoch 6/100
1/1 _____ 0s 35ms/step - loss: 0.3441 - mae: 0.5237 -
val_loss: 0.3470 - val_mae: 0.5287
Epoch 7/100
1/1 _____ 0s 35ms/step - loss: 0.3363 - mae: 0.5166 -
val_loss: 0.3388 - val_mae: 0.5217
Epoch 8/100
1/1 _____ 0s 35ms/step - loss: 0.3321 - mae: 0.5130 -
val_loss: 0.3302 - val_mae: 0.5142
Epoch 9/100
1/1 _____ 0s 38ms/step - loss: 0.3268 - mae: 0.5082 -
val_loss: 0.3209 - val_mae: 0.5060
Epoch 10/100
1/1 _____ 0s 35ms/step - loss: 0.3107 - mae: 0.4929 -
val_loss: 0.3107 - val_mae: 0.4968
Epoch 11/100
1/1 _____ 0s 45ms/step - loss: 0.3046 - mae: 0.4871 -
val_loss: 0.2997 - val_mae: 0.4866
Epoch 12/100
1/1 _____ 0s 44ms/step - loss: 0.2926 - mae: 0.4772 -
val_loss: 0.2880 - val_mae: 0.4757
Epoch 13/100
1/1 _____ 0s 37ms/step - loss: 0.2809 - mae: 0.4650 -
val_loss: 0.2756 - val_mae: 0.4636
Epoch 14/100
1/1 _____ 0s 36ms/step - loss: 0.2693 - mae: 0.4552 -
val_loss: 0.2625 - val_mae: 0.4505
Epoch 15/100
1/1 _____ 0s 37ms/step - loss: 0.2579 - mae: 0.4428 -
val_loss: 0.2487 - val_mae: 0.4361
Epoch 16/100
1/1 _____ 0s 36ms/step - loss: 0.2489 - mae: 0.4351 -
val_loss: 0.2342 - val_mae: 0.4204
Epoch 17/100
1/1 _____ 0s 34ms/step - loss: 0.2331 - mae: 0.4179 -
val_loss: 0.2188 - val_mae: 0.4030
Epoch 18/100
1/1 _____ 0s 35ms/step - loss: 0.2119 - mae: 0.3939 -
val_loss: 0.2027 - val_mae: 0.3856
Epoch 19/100
1/1 _____ 0s 34ms/step - loss: 0.1962 - mae: 0.3734 -
val_loss: 0.1858 - val_mae: 0.3695
Epoch 20/100
1/1 _____ 0s 35ms/step - loss: 0.1838 - mae: 0.3656 -
val_loss: 0.1685 - val_mae: 0.3517
Epoch 21/100
1/1 _____ 0s 36ms/step - loss: 0.1655 - mae: 0.3438 -
val_loss: 0.1508 - val_mae: 0.3318
Epoch 22/100

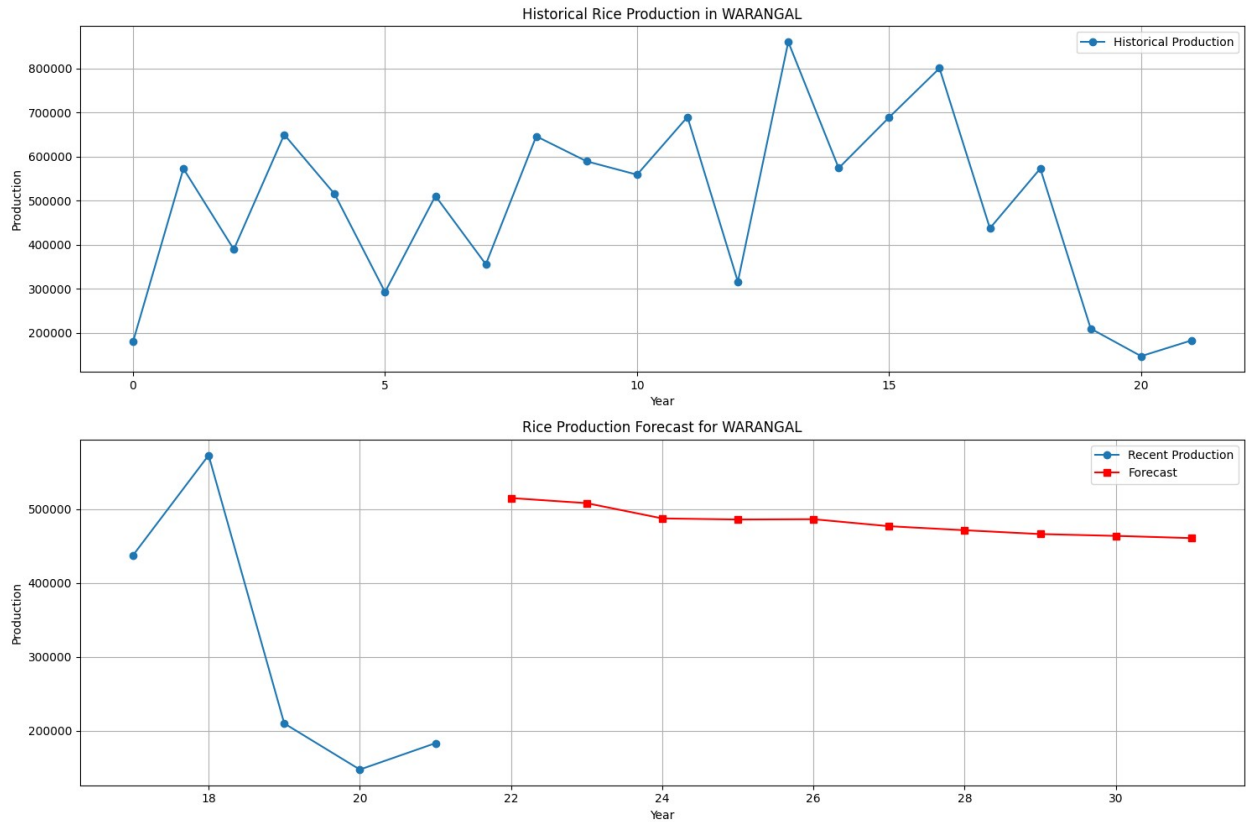
```
1/1 _____ 0s 36ms/step - loss: 0.1538 - mae: 0.3345 -  
val_loss: 0.1331 - val_mae: 0.3094  
Epoch 23/100  
1/1 _____ 0s 34ms/step - loss: 0.1315 - mae: 0.3077 -  
val_loss: 0.1158 - val_mae: 0.2865  
Epoch 24/100  
1/1 _____ 0s 34ms/step - loss: 0.1275 - mae: 0.3026 -  
val_loss: 0.0998 - val_mae: 0.2676  
Epoch 25/100  
1/1 _____ 0s 36ms/step - loss: 0.1165 - mae: 0.2913 -  
val_loss: 0.0859 - val_mae: 0.2504  
Epoch 26/100  
1/1 _____ 0s 35ms/step - loss: 0.0958 - mae: 0.2598 -  
val_loss: 0.0760 - val_mae: 0.2323  
Epoch 27/100  
1/1 _____ 0s 36ms/step - loss: 0.0912 - mae: 0.2524 -  
val_loss: 0.0720 - val_mae: 0.2213  
Epoch 28/100  
1/1 _____ 0s 34ms/step - loss: 0.0875 - mae: 0.2460 -  
val_loss: 0.0755 - val_mae: 0.2271  
Epoch 29/100  
1/1 _____ 0s 36ms/step - loss: 0.0832 - mae: 0.2224 -  
val_loss: 0.0839 - val_mae: 0.2383  
Epoch 30/100  
1/1 _____ 0s 34ms/step - loss: 0.0890 - mae: 0.2367 -  
val_loss: 0.0919 - val_mae: 0.2472  
Epoch 31/100  
1/1 _____ 0s 35ms/step - loss: 0.0907 - mae: 0.2330 -  
val_loss: 0.0957 - val_mae: 0.2514  
Epoch 32/100  
1/1 _____ 0s 35ms/step - loss: 0.1171 - mae: 0.2777 -  
val_loss: 0.0927 - val_mae: 0.2481  
Epoch 33/100  
1/1 _____ 0s 37ms/step - loss: 0.1059 - mae: 0.2604 -  
val_loss: 0.0869 - val_mae: 0.2411  
Epoch 34/100  
1/1 _____ 0s 38ms/step - loss: 0.0948 - mae: 0.2516 -  
val_loss: 0.0807 - val_mae: 0.2346  
Epoch 35/100  
1/1 _____ 0s 42ms/step - loss: 0.0847 - mae: 0.2357 -  
val_loss: 0.0760 - val_mae: 0.2280  
Epoch 36/100  
1/1 _____ 0s 35ms/step - loss: 0.0886 - mae: 0.2412 -  
val_loss: 0.0732 - val_mae: 0.2217  
Epoch 37/100  
1/1 _____ 0s 34ms/step - loss: 0.0969 - mae: 0.2474 -  
val_loss: 0.0720 - val_mae: 0.2207  
Epoch 38/100  
1/1 _____ 0s 36ms/step - loss: 0.0946 - mae: 0.2486 -
```

```
val_loss: 0.0724 - val_mae: 0.2233
Epoch 39/100
1/1 _____ 0s 35ms/step - loss: 0.0750 - mae: 0.2162 -
val_loss: 0.0740 - val_mae: 0.2274
Epoch 40/100
1/1 _____ 0s 36ms/step - loss: 0.0794 - mae: 0.2339 -
val_loss: 0.0760 - val_mae: 0.2323
Epoch 41/100
1/1 _____ 0s 35ms/step - loss: 0.0828 - mae: 0.2395 -
val_loss: 0.0779 - val_mae: 0.2367
Epoch 42/100
1/1 _____ 0s 35ms/step - loss: 0.0816 - mae: 0.2380 -
val_loss: 0.0795 - val_mae: 0.2400
Epoch 43/100
1/1 _____ 0s 35ms/step - loss: 0.0869 - mae: 0.2422 -
val_loss: 0.0804 - val_mae: 0.2417
Epoch 44/100
1/1 _____ 0s 36ms/step - loss: 0.0858 - mae: 0.2470 -
val_loss: 0.0807 - val_mae: 0.2421
Epoch 45/100
1/1 _____ 0s 35ms/step - loss: 0.1008 - mae: 0.2684 -
val_loss: 0.0805 - val_mae: 0.2419
Epoch 46/100
1/1 _____ 0s 36ms/step - loss: 0.0973 - mae: 0.2648 -
val_loss: 0.0798 - val_mae: 0.2405
Epoch 47/100
1/1 _____ 0s 34ms/step - loss: 0.0930 - mae: 0.2525 -
val_loss: 0.0788 - val_mae: 0.2385
Epoch 48/100
1/1 _____ 0s 36ms/step - loss: 0.0809 - mae: 0.2354 -
val_loss: 0.0775 - val_mae: 0.2358
Epoch 49/100
1/1 _____ 0s 37ms/step - loss: 0.0816 - mae: 0.2376 -
val_loss: 0.0761 - val_mae: 0.2326
Epoch 50/100
1/1 _____ 0s 37ms/step - loss: 0.0893 - mae: 0.2469 -
val_loss: 0.0747 - val_mae: 0.2291
Epoch 51/100
1/1 _____ 0s 37ms/step - loss: 0.0777 - mae: 0.2276 -
val_loss: 0.0735 - val_mae: 0.2262
Epoch 52/100
1/1 _____ 0s 35ms/step - loss: 0.0729 - mae: 0.2220 -
val_loss: 0.0726 - val_mae: 0.2238
Epoch 53/100
1/1 _____ 0s 37ms/step - loss: 0.0737 - mae: 0.2217 -
val_loss: 0.0721 - val_mae: 0.2224
Epoch 54/100
1/1 _____ 0s 36ms/step - loss: 0.0930 - mae: 0.2451 -
val_loss: 0.0720 - val_mae: 0.2213
```

Epoch 55/100
1/1 _____ 0s 41ms/step - loss: 0.0780 - mae: 0.2339 -
val_loss: 0.0720 - val_mae: 0.2207
Epoch 56/100
1/1 _____ 0s 47ms/step - loss: 0.0818 - mae: 0.2271 -
val_loss: 0.0722 - val_mae: 0.2207
Epoch 57/100
1/1 _____ 0s 46ms/step - loss: 0.0777 - mae: 0.2224 -
val_loss: 0.0724 - val_mae: 0.2207
Epoch 58/100
1/1 _____ 0s 40ms/step - loss: 0.0781 - mae: 0.2247 -
val_loss: 0.0725 - val_mae: 0.2207
Epoch 59/100
1/1 _____ 0s 37ms/step - loss: 0.0812 - mae: 0.2270 -
val_loss: 0.0725 - val_mae: 0.2207
Epoch 60/100
1/1 _____ 0s 35ms/step - loss: 0.0764 - mae: 0.2290 -
val_loss: 0.0724 - val_mae: 0.2207
Epoch 61/100
1/1 _____ 0s 37ms/step - loss: 0.0778 - mae: 0.2273 -
val_loss: 0.0722 - val_mae: 0.2207
Epoch 62/100
1/1 _____ 0s 36ms/step - loss: 0.0723 - mae: 0.2224 -
val_loss: 0.0721 - val_mae: 0.2207
Epoch 63/100
1/1 _____ 0s 36ms/step - loss: 0.0737 - mae: 0.2178 -
val_loss: 0.0720 - val_mae: 0.2207
Epoch 64/100
1/1 _____ 0s 37ms/step - loss: 0.0954 - mae: 0.2624 -
val_loss: 0.0719 - val_mae: 0.2207
Epoch 65/100
1/1 _____ 0s 37ms/step - loss: 0.0762 - mae: 0.2331 -
val_loss: 0.0719 - val_mae: 0.2207
Epoch 66/100
1/1 _____ 0s 36ms/step - loss: 0.0740 - mae: 0.2253 -
val_loss: 0.0719 - val_mae: 0.2210
Epoch 67/100
1/1 _____ 0s 34ms/step - loss: 0.0757 - mae: 0.2235 -
val_loss: 0.0719 - val_mae: 0.2214
Epoch 68/100
1/1 _____ 0s 35ms/step - loss: 0.0774 - mae: 0.2284 -
val_loss: 0.0720 - val_mae: 0.2218
Epoch 69/100
1/1 _____ 0s 35ms/step - loss: 0.0733 - mae: 0.2254 -
val_loss: 0.0720 - val_mae: 0.2221
Epoch 70/100
1/1 _____ 0s 36ms/step - loss: 0.0818 - mae: 0.2391 -
val_loss: 0.0721 - val_mae: 0.2223
Epoch 71/100


```
1/1 _____ 0s 36ms/step - loss: 0.0876 - mae: 0.2400 -  
val_loss: 0.0722 - val_mae: 0.2226  
Epoch 72/100  
1/1 _____ 0s 35ms/step - loss: 0.0821 - mae: 0.2330 -  
val_loss: 0.0723 - val_mae: 0.2230  
Epoch 73/100  
1/1 _____ 0s 34ms/step - loss: 0.0734 - mae: 0.2259 -  
val_loss: 0.0725 - val_mae: 0.2235  
Epoch 74/100  
1/1 _____ 0s 36ms/step - loss: 0.0734 - mae: 0.2247 -  
val_loss: 0.0726 - val_mae: 0.2237  
Epoch 75/100  
1/1 _____ 0s 35ms/step - loss: 0.0768 - mae: 0.2259 -  
val_loss: 0.0725 - val_mae: 0.2237  
Epoch 76/100  
1/1 _____ 0s 52ms/step - loss: 0.0813 - mae: 0.2379 -  
val_loss: 0.0724 - val_mae: 0.2234  
Epoch 77/100  
1/1 _____ 0s 36ms/step - loss: 0.0889 - mae: 0.2453 -  
val_loss: 0.0724 - val_mae: 0.2233  
Epoch 78/100  
1/1 _____ 0s 37ms/step - loss: 0.0907 - mae: 0.2524 -  
val_loss: 0.0723 - val_mae: 0.2229  
Epoch 79/100  
1/1 _____ 0s 35ms/step - loss: 0.0846 - mae: 0.2336 -  
val_loss: 0.0722 - val_mae: 0.2227  
Epoch 80/100  
1/1 _____ 0s 37ms/step - loss: 0.0847 - mae: 0.2385 -  
val_loss: 0.0721 - val_mae: 0.2223  
Epoch 81/100  
1/1 _____ 0s 34ms/step - loss: 0.0877 - mae: 0.2495 -  
val_loss: 0.0719 - val_mae: 0.2217  
Epoch 82/100  
1/1 _____ 0s 35ms/step - loss: 0.0794 - mae: 0.2318 -  
val_loss: 0.0719 - val_mae: 0.2212  
Epoch 83/100  
1/1 _____ 0s 36ms/step - loss: 0.0666 - mae: 0.2147 -  
val_loss: 0.0719 - val_mae: 0.2208  
Epoch 84/100  
1/1 _____ 0s 36ms/step - loss: 0.0748 - mae: 0.2246 -  
val_loss: 0.0719 - val_mae: 0.2208  
Epoch 85/100  
1/1 _____ 0s 36ms/step - loss: 0.0816 - mae: 0.2306 -  
val_loss: 0.0719 - val_mae: 0.2208  
Epoch 86/100  
1/1 _____ 0s 35ms/step - loss: 0.0749 - mae: 0.2269 -  
val_loss: 0.0719 - val_mae: 0.2208  
Epoch 87/100  
1/1 _____ 0s 35ms/step - loss: 0.0760 - mae: 0.2260 -
```

```
val_loss: 0.0719 - val_mae: 0.2208
Epoch 88/100
1/1 _____ 0s 35ms/step - loss: 0.0782 - mae: 0.2295 -
val_loss: 0.0719 - val_mae: 0.2208
Epoch 89/100
1/1 _____ 0s 35ms/step - loss: 0.0755 - mae: 0.2274 -
val_loss: 0.0719 - val_mae: 0.2209
Epoch 90/100
1/1 _____ 0s 36ms/step - loss: 0.0775 - mae: 0.2292 -
val_loss: 0.0719 - val_mae: 0.2209
Epoch 91/100
1/1 _____ 0s 35ms/step - loss: 0.0803 - mae: 0.2310 -
val_loss: 0.0719 - val_mae: 0.2209
Epoch 92/100
1/1 _____ 0s 35ms/step - loss: 0.0775 - mae: 0.2296 -
val_loss: 0.0719 - val_mae: 0.2211
Epoch 93/100
1/1 _____ 0s 35ms/step - loss: 0.0717 - mae: 0.2191 -
val_loss: 0.0719 - val_mae: 0.2215
Epoch 94/100
1/1 _____ 0s 39ms/step - loss: 0.0746 - mae: 0.2297 -
val_loss: 0.0720 - val_mae: 0.2217
Epoch 95/100
1/1 _____ 0s 44ms/step - loss: 0.0849 - mae: 0.2358 -
val_loss: 0.0720 - val_mae: 0.2221
Epoch 96/100
1/1 _____ 0s 40ms/step - loss: 0.0759 - mae: 0.2260 -
val_loss: 0.0721 - val_mae: 0.2225
Epoch 97/100
1/1 _____ 0s 34ms/step - loss: 0.0788 - mae: 0.2283 -
val_loss: 0.0722 - val_mae: 0.2228
Epoch 98/100
1/1 _____ 0s 35ms/step - loss: 0.0745 - mae: 0.2288 -
val_loss: 0.0722 - val_mae: 0.2229
Epoch 99/100
1/1 _____ 0s 36ms/step - loss: 0.0816 - mae: 0.2386 -
val_loss: 0.0723 - val_mae: 0.2230
Epoch 100/100
1/1 _____ 0s 36ms/step - loss: 0.0750 - mae: 0.2201 -
val_loss: 0.0722 - val_mae: 0.2227
```



Production Forecast:

Year 22: 514,976
 Year 23: 508,092
 Year 24: 487,405
 Year 25: 485,912
 Year 26: 486,327
 Year 27: 476,857
 Year 28: 471,398
 Year 29: 466,215
 Year 30: 463,747
 Year 31: 460,757

Climate Correlation Analysis:

YEAR: -0.074
 monsoon_rainfall: 0.305
 pre_monsoon: -0.379
 post_monsoon: -0.111
 annual_temp: -0.017
 temp_variation: 0.214

Model Training Performance:

Final loss: 0.0750
 Final MAE: 0.2201

